



Operations Research

Publication details, including instructions for authors and subscription information:
<http://pubsonline.informs.org>

Scheduling with Finite Capacity Output Buffers

Nicholas G. Hall, Marc E. Posner, Chris N. Potts,

To cite this article:

Nicholas G. Hall, Marc E. Posner, Chris N. Potts, (1998) Scheduling with Finite Capacity Output Buffers. *Operations Research* 46(3-supplement-3):S84-S97. <https://doi.org/10.1287/opre.46.3.S84>

Full terms and conditions of use: <https://pubsonline.informs.org/Publications/Librarians-Portal/PubsOnLine-Terms-and-Conditions>

This article may be used only for the purposes of research, teaching, and/or private study. Commercial use or systematic downloading (by robots or other automatic processes) is prohibited without explicit Publisher approval, unless otherwise noted. For more information, contact permissions@informs.org.

The Publisher does not warrant or guarantee the article's accuracy, completeness, merchantability, fitness for a particular purpose, or non-infringement. Descriptions of, or references to, products or publications, or inclusion of an advertisement in this article, neither constitutes nor implies a guarantee, endorsement, or support of claims made of that product, publication, or service.

© 1998 INFORMS

Please scroll down for article—it is on subsequent pages



With 12,500 members from nearly 90 countries, INFORMS is the largest international association of operations research (O.R.) and analytics professionals and students. INFORMS provides unique networking and learning opportunities for individual professionals, and organizations of all types and sizes, to better understand and use O.R. and analytics tools and methods to transform strategic visions and achieve better outcomes.

For more information on INFORMS, its publications, membership, or meetings visit <http://www.informs.org>

SCHEDULING WITH FINITE CAPACITY OUTPUT BUFFERS

NICHOLAS G. HALL and MARC E. POSNER

The Ohio State University, Columbus, Ohio

CHRIS N. POTTS

University of Southampton, Southampton, England

(Received March 1994; revisions received July 1995, March 1996; accepted November 1996)

In many scheduling problems, a job that completes processing may need to be held in an output buffer until the customer is ready to accept delivery. Buffer capacity is usually assumed to be infinite.

We study a number of the best known single machine scheduling problems, under several alternative assumptions about the capacity of the output buffer. Specifically, we allow the buffer capacity to be either zero, fixed, or specified as part of problem input. We also consider situations in which all jobs have the same storage requirement in the buffer, and others where the storage requirement may vary. Further, we consider generalizations where there is a time interval within which a customer accepts delivery without cost to the producer.

A classification scheme for these problems is provided. For each problem considered, we provide either an efficient algorithm or a proof that such an algorithm is unlikely to exist. Our results provide a mapping of the computational complexity of these problems which parallels those that are available for classical scheduling problems with infinite buffer capacity.

We consider a variant of the classical deterministic scheduling environment, where decisions involving *output buffers* are important. Consider a situation in which n jobs are to be processed nonpreemptively on a single machine, and where a job has been completed for a customer who is not yet ready to receive it. The job is held in an output buffer from the time at which it completes processing until the customer is ready to accept delivery. The usual assumption is that the output buffer has infinite capacity.

In this paper we make the assumption that the capacity of the output buffer is finite. In particular, we consider three cases relating to buffer capacity:

1. no buffer exists;
2. buffer capacity is a given constant;
3. buffer capacity is specified as part of problem input.

In each of these environments, we study several of the best known single machine scheduling problems. For each problem, we provide an efficient algorithm or an *NP*-completeness proof. Among the most natural and important objectives are minimizing the maximum completion time (makespan) of the schedule, minimizing the number of jobs not delivered to the customers by their due dates, and minimizing the maximum lateness of the jobs.

Scheduling problems with finite capacity output buffers arise in many types of manufacturing environments. Consider, for example, production processes with physical space limitations. Finite capacity output buffers model limited warehouse space for finished goods. Apart from unavoidable physical space constraints, there are several

reasons why a manufacturing company may wish to limit buffer capacities. One reason is that holding products in buffers can be expensive. Not only are there costs of insurance, record-keeping and deterioration, but the opportunity cost of financing the inventory is often considerable (McClain and Thomas 1985). Another reason relates to plant layout. The factory of the future is designed in a space-efficient way. It has already become customary in Japan for physical space in factories to be severely constrained (Gaither 1990). A key element in making a plant space-efficient is the reduction of space allowed for buffers of all types: input buffers for the storage of jobs prior to processing, intermediate buffers between machines, and output buffers.

Further motivations for limiting buffer space relate to quality of service to customers, and to the type of product being manufactured. In practice, buffers have a tendency to become full as a result of poor shop floor practices (Weiss and Gershon 1993). If large buffers are allowed to fill, then some products may be spending longer than necessary in the shop. Not only does this increase inventory costs, but it can also degrade the service to customers. This degradation can occur, for example, through product deterioration or inventory management problems. Finally, there are specific products that cannot be held for a long time (sometimes, not at all) in buffers. These include chemical, steel and aluminum products (Reklaitis 1982, Hall and Sriskandarajah 1996), where the chemical state or temperature of the product must be maintained to prevent degradation. Limiting buffer *space* can be a good proxy for limiting the time that products are held in the buffer.

Subject classifications: Production/scheduling: scheduling with limited output storage capacity. Deterministic, single machine sequencing.

Area of review: MANUFACTURING.

Buffer capacity issues also arise in the food preparation industry (Ramudhin and Ratliff 1991). As an everyday example, McDonald's makes hamburgers to stock, rather than to order. After preparation, the hamburgers wait under heat lamps in a finite capacity output buffer. If the buffer capacity were increased, it is likely that the quality of the product (i.e., freshness) would be diminished.

There are only a few studies of deterministic scheduling problems involving finite buffers, all of which consider only the input buffer case. Nawijn (1992) shows that, if the processing order on the machine is restricted to the order in which the jobs arrive, the problem of minimizing the total weight of jobs lost on arrival can be solved in polynomial time. In related work, Nawijn et al. (1994) provide a polynomial time algorithm for the problem of minimizing the number of jobs lost on arrival, where buffer capacity is a constant. However, they also show that minimizing a weighted version of this objective is binary *NP*-hard. Hall et al. (1998) present a computational complexity classification of various single machine scheduling problems with finite capacity input buffers. A similar study of problems in which job preemptions are permitted is given by Hall et al. (1997).

There is some research that is applicable for problems with zero buffer capacity. The work of Arkin and Silverberg (1987) on scheduling with fixed start and end times is relevant for the weighted number of late jobs problem. Also, Hoogeveen (1996) provides an algorithm that can be applied to the maximum lateness problem with zero buffer capacity.

In Section 1, we provide a scheme for the classification of scheduling problems with finite capacity output buffers, and present an overview of our results. The next three sections give algorithms and computational complexity results for problems with finite capacity output buffers. The results are organized by optimization criteria. For each criterion, we proceed from the most restrictive to the most general problem. In Section 2, we consider the makespan and the sum of dispatch times problems. Problems involving the number of late jobs are examined in Section 3. In Section 4, we consider lateness and tardiness problems. Section 5 contains some conclusions and suggestions for future work.

1. PRELIMINARIES

1.1. Notation and Classification

We begin with some notation. Let $N = \{1, \dots, n\}$ denote the set of jobs. Let p_j denote the processing time and w_j the weight (or value) of job j for $j \in N$, and let $W = \sum_{j=1}^n w_j$. The buffer storage requirement of job j is denoted by s_j , and the buffer capacity is denoted by b_O . The earliest time at which job j can be dispatched is denoted by e_j . In some of the problems studied, job j has a due date d_j , where $d_j \geq e_j$. The due date is defined to be the latest time at which the job can be dispatched

in order to arrive at the customer on time. We assume throughout that p_j , w_j , s_j , e_j , and d_j are integer, for $j \in N$. For any schedule σ , we also define:

$C_j(\sigma)$ = the time at which job j is dispatched to its customer;

$U_j(\sigma) = \begin{cases} 1 & \text{if job } j \text{ is late,} \\ 0 & \text{if job } j \text{ is dispatched to its customer} \\ & \text{by its due date;} \end{cases}$

$T_j(\sigma) = \max\{C_j(\sigma) - d_j, 0\}$, the tardiness of job j .

Each job is dispatched to a customer at the later of its earliest dispatch time and the time at which it finishes processing. It is appropriate to base our performance measures on the dispatch times of jobs rather than on the times at which they complete processing, because the dispatch time is the completion time as perceived by the customer. When there is no ambiguity, we simplify $C_j(\sigma)$, $U_j(\sigma)$, and $T_j(\sigma)$ to C_j , U_j , and T_j , respectively.

We use the convention that the *scheduling* of a job is a decision about the time interval in which to process the job. Since the processing of a job can be delayed, we assume without loss of generality that the machine cannot be used for storage. After a job completes processing, it is removed from the machine and either dispatched to the customer or put in an output buffer.

The standard classification scheme for scheduling problems (Graham et al. 1979) is $\psi_1|\psi_2|\psi_3$, where ψ_1 indicates the scheduling environment, ψ_2 describes the job characteristics or restrictive requirements, and ψ_3 defines the objective function to be minimized. Here we extend this scheme to provide for the classification of scheduling problems with finite capacity output buffers, as follows.

Under ψ_2 , we may have

$b_O = 0$: output buffer capacity is equal to zero;

$b_O = b$: output buffer capacity is a nonzero constant;

b_O : output buffer capacity is specified as part of problem input;

$e_j = d_j$: the earliest dispatch time of every job is equal to its due date;

$s_j = 1$: all jobs have a unit buffer storage requirement.

Since we are interested in analyzing the simplest problems with buffers, we do not examine output buffer problems with release dates. However, if $b_O = 0$, then job j has a de facto release date of $r_j = e_j - p_j$.

We note that in some of the problems, an optimal assignment of jobs to the buffer is similar to solving a knapsack problem. However, when $s_j = 1$, the buffer size specifies the number of jobs that the buffer can hold, and any b_O items fill the buffer. When $b_O = 0$, there is no need to specify s_j .

The objective functions that we consider under ψ_3 are as follows:

Table I
The Complexity of Output Buffer Problems

Objective	Buffer Type	$e_j = d_j$	$s_j = 1$	Complexity Results	
$C_{\max}$	$b_O = 0$	n.a.	n.a.	$O(n \log n)$	Thm 2.1
	$b_O = b$	n.a.	Yes	$O(n \log n)$	Thm 2.2
			No	$O(n^{b+3})$	Thm 2.3
	b_O	n.a.	Yes	$O(n \log n)$	Thm 2.2
ΣC_j	Any	n.a.	No	Unary NPC	Thm 2.4
			Any	Unary NPC	Thm 2.5, 2.6
ΣU_j	$b_O = 0$	Yes	n.a.	$O(n \log n)$	Thm 3.1
		No	n.a.	Unary NPC	Thm 3.2
	$b_O = b$	Yes	Any	$O(n^{b+2})$	Cor 3.1
		No	Any	Unary NPC	Thm 3.5
	b_O	Any	Any	Unary NPC	Thm 3.6
				Unary NPC	Thm 3.6
$\Sigma w_j U_j$	$b_O = 0$	Yes	n.a.	$O(n^2)$	Arkin and Silverberg (1987)
		No	n.a.	Unary NPC	Thm 3.2
	$b_O = b$	Yes	Any	Binary NPC, $O(Wn^{b+1})$	Thm 3.4, 3.3
		No	Any	Unary NPC	Thm 3.5
	b_O	Any	Any	Unary NPC	Thm 3.6
				Unary NPC	Thm 3.6
$L_{\max}$	$b_O = 0$	Yes	n.a.	$O(n^2 \log n)$	Hoogeveen (1996)
		No	n.a.	Unary NPC	Thm 4.1
	$b_O = b$	Yes	Any	$O(n^{2b+4})$	Thm 4.2
		No	Any	Unary NPC	Cor 4.1
	b_O	Any	Any	Unary NPC	Cor 4.2
				Unary NPC	Cor 4.2
ΣT_j	Any	Any	Any	Unary NPC	Thm 4.3

$C_{\max} = \max_{j \in N} \{C_j\}$, the dispatch time of the last job, or the makespan;

ΣC_j = the total dispatch time of the jobs;

ΣU_j = the number of jobs not dispatched by their due dates;

$\Sigma w_j U_j$ = the total weight of jobs not dispatched by their due dates;

$L_{\max} = \max_{j \in N} \{C_j - d_j\}$, the maximum lateness of the jobs;

ΣT_j = the total tardiness of the jobs.

An example of the classification scheme is problem $1|b_O = b, e_j = d_j|L_{\max}$, which denotes the minimization of the maximum lateness on a single machine with an output buffer having nonzero, constant capacity, and jobs having earliest dispatch times equal to due dates, but possibly different buffer storage requirements.

1.2. Overview of the Results

Because the cost and the feasibility of a schedule can be evaluated in $O(n)$ time, the recognition versions of all the problems considered in this paper belong to the class NP.

Table I presents the results of this paper. The entry

“Any” in the Buffer Type column implies that the result holds for any of the three buffer types: $b_O = 0$, $b_O = b$, and b_O . A “Yes” in the $e_j = d_j$ column implies that the earliest dispatch time is equal to the due date, “No” implies that the earliest dispatch time may be less than the due date, and “Any” implies either Yes or No. Where a specification is not applicable, the relevant cell is denoted “n.a.”. In the $s_j = 1$ column, the “Yes” and “No” are similarly defined. “Any” in the $s_j = 1$ column means either Yes or No if the buffer type is $b_O = b$ or b_O , and it means not applicable if the buffer type is $b_O = 0$. We use Unary NPC and Binary NPC to indicate that the equivalent recognition version of a problem is NP-complete with respect to a unary and binary encoding of the data, respectively. Related definitions can be found in Garey and Johnson (1979). Included in the appropriate cell is a reference to where a proof of that result can be found.

It is natural to ask whether scheduling problems with finite buffers are easier or harder than their counterparts with infinite buffers. Our results show that no overall conclusion is possible. To see this, consider the classical scheduling problem of minimizing the weighted number of late jobs, $1||\Sigma w_j U_j$. This problem is binary NP-hard (Karp 1972), and it can be solved by the pseudo-polynomial time algorithm proposed by Lawler and Moore (1969). However,

Table I shows that if $b_O = 0$ and $e_j = d_j$, then this problem is solvable in $O(n^2)$ time; whereas the arbitrary b_O case is unary NP-hard.

1.3. Output Buffers

If a job finishes processing before its earliest dispatch time, then it is stored in the output buffer until it can be sent to the customer. However, the storage requirement of jobs already in the buffer may be sufficiently large that no other job can enter the buffer until one or more jobs leave. In this case, machine idle time occurs. Nevertheless, there is no advantage to delaying unnecessarily the next job to be processed. Thus, we restrict our attention to schedules in which, after a period of idle time, processing of the next job starts as early as possible.

Let B denote the set of jobs that occupy the buffer at some given time. We refer to B as a *buffer configuration*. Let $\mathcal{B}$ denote the set of all feasible buffer configurations.

We define job k to be *available* at some specified time t if $t \geq e_k - p_k$, and k has not started processing at that time. If the machine is free and job k is available at time t , then k can be completed and dispatched at time $t + p_k$.

We now derive an expression for the earliest time that a given job k can complete processing, given that, at time t , the machine becomes free and the buffer configuration is B . If $t + p_k \geq e_k$, then job k has an earliest completion time of $t + p_k$. Now, suppose that $t + p_k < e_k$. If job k starts processing at time t , then upon completion of processing, k must be stored in the buffer. If $B \cup \{k\} \in \mathcal{B}$, then there is sufficient space in the buffer to store job k . Thus, k can complete processing at time $t + p_k$. Alternatively, suppose that $B \cup \{k\} \notin \mathcal{B}$. Then there is insufficient space in the buffer to store job k at time $t + p_k$, and a period of idle time must be inserted before processing job k . We define f to be the first job to leave the buffer after time t that creates sufficient space for job k to be stored. More formally, let $f \in B$, where e_f is as small as possible, so that $s_k + \sum_{j \in \{i \in B, e_i > e_f\}} s_j \leq b_O$. When f does not exist, we let $e_f = \infty$. If $e_k \leq e_f$, then the earliest time that job k can complete processing is e_k . Alternatively, if $e_k > e_f$, then job k can complete processing at time $\max\{t + p_k, e_f\}$. Combining these observations, the earliest time that job k can complete processing is

$$E_k(t, B) = \begin{cases} t + p_k & \text{if } B \cup \{k\} \in \mathcal{B}, \\ \max\{t + p_k, \min\{e_k, e_f\}\} & \text{otherwise.} \end{cases} \quad (1)$$

For $b_O = b$, the value $E_k(t, B)$ can be computed in constant time.

2. DISPATCH TIME PROBLEMS

For each objective, we start by analyzing problems with unit storage requirements, and then generalize the results to the case where storage requirements may vary between the jobs. We begin our study of finite capacity output buffers with the problems of minimizing the makespan and of minimizing the sum of dispatch times.

2.1. Makespan

We begin by presenting an optimality condition for all makespan problems.

Lemma 2.1. *For any makespan problem, there is an optimal schedule in which job i starts processing at the first time that the machine is free and job i is available.*

Proof. Let σ^* denote an optimal schedule in which job i is available at time t . Suppose job i does not start processing until time u in σ^* , where $u > t$. Then, we construct a schedule σ from σ^* by starting the processing of job i at time t and by delaying all jobs that start at or after time t but before time u in σ^* by p_i units of time. Because i is available at time t , for all t' where $t \leq t' \leq u$, the jobs in the buffer at time $t' + p_i$ in σ are a subset of those in the buffer at time t' in σ^* . Thus, schedule σ is feasible. Since the last job in σ is processed no later than that in σ^* , we conclude that σ is also an optimal schedule. $\square$

For problem 1| $b_O = 0$ | $C_{\max}$, the absence of buffer space does not allow any job j to begin processing earlier than time $e_j - p_j$. If we let $r_j = e_j - p_j$ for $j \in N$, where r_j is the release date for job j , then the problem is equivalent to 1| r_j | $C_{\max}$. It can be seen that a solution to this problem is to sequence the jobs in nondecreasing r_j order, and then process each job as soon as the job and the machine become available. We have established the following result.

Theorem 2.1. *An optimal schedule for problem 1| $b_O = 0$ | $C_{\max}$ can be found in $O(n \log n)$ time by sequencing jobs in nondecreasing order of $e_j - p_j$.*

We present an algorithm for cases of the makespan problem with $s_j = 1$, and nonzero buffer capacity. In this algorithm, we first process all available jobs in accordance with Lemma 2.1. When there is no available job, a longest unscheduled job is processed, provided that upon completion, there is space in the buffer to put that job. The processing of available and longest jobs continues until it is impossible to schedule a job without having a period of machine idle time. Following the machine idle time, the next job processed is one that can be started earliest among the unscheduled jobs. This is either an unscheduled job q for which $e_q - p_q$ is as small as possible, or an unscheduled job l with the longest processing time. Job q can start processing at time $e_q - p_q$, and l can start processing at time $e_k - p_l$, where k is the first job that is dispatched from the buffer after the idle time period.

In the formal statement of our algorithm, σ is the current partial schedule, t is the time at which processing of σ completes, B is the set of jobs in the buffer at time t , and Q is the set of unscheduled jobs.

Algorithm BIGFILL

Initialization

Set $t = 0$, $Q = N$, $B = \emptyset$ and $\sigma = \emptyset$.

Available Job Scheduling

If there is a job i such that $i \in Q$ and $e_i - p_i \leq t$, then set $Q = Q \setminus \{i\}$, $t = t + p_i$, $B = B \setminus \{j \mid j \in B, e_j \leq t\}$ and $\sigma = (\sigma, i)$, and go to Termination Test.

Longest Job Scheduling

Set $l = \operatorname{argmax}\{p_j \mid j \in Q\}$ and $k = \operatorname{argmin}\{e_j \mid j \in B\}$. If $|B| < b_O$ or $t + p_l \geq e_k$, then set $Q = Q \setminus \{l\}$, $t = t + p_l$, $B = B \setminus \{j \mid j \in B, e_j \leq t\}$, $\sigma = (\sigma, l)$ and $B = B \cup \{l\}$, and go to Termination Test.

Earliest Start Time Scheduling

Set $q = \operatorname{argmin}\{e_j - p_j \mid j \in Q\}$. If $e_k - p_l < e_q - p_q$, then set $Q = Q \setminus \{l\}$, $t = e_k$, $B = B \cup \{l\} \setminus \{j \mid j \in B, e_j \leq t\}$ and $\sigma = (\sigma, l)$. Otherwise, set $Q = Q \setminus \{q\}$, $t = e_q$, $B = B \setminus \{j \mid j \in B, e_j \leq t\}$ and $\sigma = (\sigma, q)$.

Termination Test

If $Q \neq \emptyset$, then go to Available Job Scheduling.

Output

Output σ and $\max\{t, \max_{j \in N} \{e_j\}\}$.

We now analyze the performance of Algorithm BIGFILL.

Theorem 2.2. *Algorithm BIGFILL generates an optimal schedule for problems $1|b_O = b, s_j = 1|C_{\max}$ and $1|b_O, s_j = 1|C_{\max}$ in $O(n \log n)$ time.*

Proof. Let t be the end of some idle time period in a schedule σ generated by Algorithm BIGFILL. To prove that σ is optimal, it is sufficient to show that, for every t , no other schedule has less idle time in the interval $[0, t]$.

Let B be the jobs in the buffer at time t in σ . It follows from the Longest Job Scheduling step and the fact that the machine is idle during $[t - 1, t]$, that $|B| = b_O$. Let $A = \{j \mid j \in N, e_j \leq t\}$. Note that the Earliest Start Time Scheduling step ensures that all jobs of A are processed by time t . Consequently, the total processing time during $[0, t]$ in σ is $\sum_{j \in A \cup B} p_j$.

Consider any other feasible schedule σ' that has completed the processing of jobs $A' \cup B'$ in the interval $[0, t]$, where $A' \subseteq A$, and where $B' \subseteq N \setminus A$ are the jobs in the buffer at time t . The buffer capacity constraint implies that $|B'| \leq b_O$.

First, assume that the machine is idle during $[t - 1, t]$ in schedule σ' . Observe that since there is idle time in σ , the Longest Job Scheduling step ensures that the jobs with the longest processing times are scheduled. Thus, $\sum_{j \in B} p_j \geq \sum_{j \in B'} p_j$. Since $A' \subseteq A$, we conclude that $\sum_{j \in A \cup B} p_j \geq \sum_{j \in A' \cup B'} p_j$, and σ has no more idle time than σ' in the interval $[0, t]$.

Now we consider the alternative assumption that in schedule σ' , some job h is processed in the interval $[t - 1, t]$. If $h \in A$, then the Longest Job Scheduling step ensures that $\sum_{j \in B} p_j \geq \sum_{j \in B'} p_j$. Since $A' \cup \{h\} \subseteq A$, it follows that $\sum_{j \in A \cup B} p_j \geq \sum_{j \in A' \cup B' \cup \{h\}} p_j$, and σ has no more idle time than σ' in the interval $[0, t]$.

We now consider the case where $h \notin A$. Suppose that $|B'| < b_O$ or $h \in B'$. Because $h \notin A$, h is a candidate in

the Longest Job Scheduling step. For this case, we have that $\sum_{j \in B} p_j \geq \sum_{j \in B' \cup \{h\}} p_j$. Since $A' \subseteq A$, it follows that $\sum_{j \in A \cup B} p_j \geq \sum_{j \in A' \cup B' \cup \{h\}} p_j$. Thus, when $|B'| < b_O$ or $h \in B'$, we conclude that σ has no more idle time than σ' in the interval $[0, t]$.

Consequently, we suppose that $|B'| = b_O = |B|$ and $h \notin B'$. This implies that either h is dispatched upon completion or at least one job of B' has left the buffer by the time that h completes processing. Thus, there is some job i , where either $i = h$ or $i \in B'$, such that job h completes processing no earlier than time e_i .

There are two cases, $i \in B$ and $i \notin B$. In each case, we show that σ has no more idle time than σ' in the interval $[0, e_i]$. Since the machine processes job h throughout the interval $[t, e_i]$ in σ' , this establishes the required result that σ has no more idle time than σ' in the interval $[0, t]$.

Case 1. $i \in B$. Since i is dispatched at time e_i in schedule σ , the Earliest Start Time Scheduling step ensures that the total processing time in the interval $[0, e_i]$ in σ is at least $\sum_{j \in A \cup B} p_j + \max_{j \in N \setminus (A \cup B)} \{p_j\}$. The total processing time in the interval $[0, e_i]$ in σ' is at most $\sum_{j \in A' \cup B'} p_j + p_h$. The Longest Job Scheduling step and the fact that $|B| = |B'|$ ensure that $\sum_{j \in B} p_j + \max_{j \in N \setminus (A \cup B)} \{p_j\} \geq \sum_{j \in B'} p_j + p_h$. Since $A' \subseteq A$, we conclude that $\sum_{j \in A \cup B} p_j + \max_{j \in N \setminus (A \cup B)} \{p_j\} \geq \sum_{j \in A' \cup B'} p_j + p_h$. This shows that σ has no more idle time than σ' in the interval $[0, e_i]$.

Case 2. $i \notin B$. Since, in σ , $i \notin B$ and the machine is idle during the interval $[t - 1, t]$, the Earliest Start Time Scheduling step ensures that $e_i - p_i \geq t$. Because job i cannot begin processing before time t and complete processing at time e_i or later, we deduce that $i \neq h$. Consequently, $i \in B' \setminus B$. Since σ can start processing job i at time $e_i - p_i$, it follows from the Earliest Start Time Scheduling step that the total processing time in the interval $[0, e_i]$ in σ is at least $\sum_{j \in A \cup B \cup \{i\}} p_j$. The total processing time in the interval $[0, e_i]$ in σ' is at most $\sum_{j \in A' \cup B'} p_j + p_h$. The Longest Job Scheduling step ensures that $\sum_{j \in B} p_j \geq \sum_{j \in B' \setminus \{i\}} p_j + p_h$. Since $i \in B' \setminus B$ and $A' \subseteq A$, we have that $\sum_{j \in A \cup B \cup \{i\}} p_j \geq \sum_{j \in A' \cup B'} p_j + p_h$. Thus, σ has no more idle time than σ' in the interval $[0, e_i]$.

We have established that Algorithm BIGFILL generates an optimal schedule. Now, we derive its time complexity. To implement BIGFILL efficiently, we create three heaps: one for the jobs of Q using p_j as a key, one for the jobs of Q using $e_j - p_j$ as a key, and one for the jobs of B using e_j as a key. Since each job is removed from Q once, and each job is inserted and removed from B at most once, these heaps can be created and maintained in $O(n \log n)$ time. Because each job is processed once, the computations associated with selecting a job to schedule next, and updating t and σ , require $O(n)$ time. Thus, the time complexity of Algorithm BIGFILL is $O(n \log n)$. $\square$

Next, we consider problem $1|b_O = b|C_{\max}$. Although Algorithm BIGFILL generates an optimal schedule for the corresponding problem when $s_j = 1$, the following example

shows that this procedure does not necessarily provide an optimal schedule for the more general problem.

Example. Let $n = 3$; $p_1 = p_2 = 2$, $p_3 = 4$; $e_1 = e_2 = e_3 = 8$; $s_1 = s_2 = 2$, $s_3 = 3$; $b_O = 4$. Algorithm BIGFILL processes job 3 from time 0 to 4. No new jobs can enter the buffer until time 8. Idle time is necessary from time 4 to 6, when jobs 1 and then 2 can begin processing. This schedule has a makespan of 10. An optimal schedule processes job 1 from time 0 to 2, job 2 from time 2 to 4, and job 3 from time 4 to 8, giving a makespan of 8.

Problem $1|b_O = b|C_{\max}$ can still be solved in polynomial time by using a dynamic programming algorithm. Each iteration of this algorithm first appends a job to one of the current partial schedules. Then, Lemma 2.1 is used to add further jobs to the end of the partial schedule until there are no available jobs that can be appended. We say that such a partial schedule, where there are no available jobs, is *nonextendable*. Due to the important role of Lemma 2.1 in our dynamic programming algorithm, we reindex the jobs so that $e_1 - p_1 \leq \dots \leq e_n - p_n$.

The states of the dynamic programming algorithm are (v, B) . In the corresponding (nonextendable) partial schedule, v is the number of jobs that have been processed and B is the buffer configuration when the partial schedule completes processing. The dynamic programming algorithm recursively computes values $\Omega(v, B)$, which represent the minimum completion time of all partial schedules that have been generated thus far and achieve the state (v, B) .

Before proceeding with a description of our algorithm, we present two preliminary results. The first of these shows that knowledge of the state (v, B) is sufficient to determine which jobs appear in the corresponding nonextendable partial schedule. The second result establishes that the elimination criterion for the dynamic programming algorithm is valid.

Lemma 2.2. *In any nonextendable partial schedule corresponding to state (v, B) , the set of processed jobs is $R = \{1, \dots, h\} \cup B$, where $h = v - |B|$.*

Proof. Let t denote the time at which the partial schedule corresponding to state (v, B) completes processing, and let A denote the set of jobs that are processed and dispatched by time t . Since $|B|$ jobs are in the buffer at time t , we have that $|A| = h$. If $A = \emptyset$, then $R = B$ and the desired result is established.

Alternatively, if $A \neq \emptyset$, then there is a job $k \in A$, where $k \geq h$. Thus, the partial schedule completes processing no earlier than time $e_k > e_h - p_k \geq e_h - p_h$. Let job $j = \min\{i | i \in N(A \cup B)\}$. If $j \leq h$, then $e_j - p_j \leq e_h - p_h$, and job j is available when the partial schedule completes processing. This contradicts the property that the schedule is nonextendable. It follows that the set of dispatched jobs is $\{1, \dots, h\}$ and the processed jobs are defined by the set R . $\square$

Lemma 2.3. *Suppose two nonextendable partial schedules, π_1 and π_2 , reach the same state (v, B) . Further suppose that π_1 and π_2 complete processing at this state at times $M(\pi_1)$ and $M(\pi_2)$, respectively, where $M(\pi_1) \leq M(\pi_2)$. Then, in the search for an optimal schedule, π_2 can be discarded.*

Proof. From Lemma 2.2, the sets of jobs that have been processed in the two partial schedules are identical. Thus, any feasible continuation of the second partial schedule is a feasible continuation of the first. Consequently, the first partial schedule is preferred. $\square$

Lemma 2.3 provides an elimination criterion which only applies to nonextendable partial schedules. Whenever a job is appended to a partial schedule in our dynamic programming algorithm, Lemma 2.1 is used repeatedly to create a nonextendable partial schedule. Thus, by applying Lemma 2.3, this nonextendable partial schedule may possibly be eliminated or used to eliminate other nonextendable partial schedules. We now give a formal statement of the dynamic programming algorithm for problem $1|b_O = b|C_{\max}$.

Algorithm Makespan Dynamic Program (MDP)

Initialization

Set $\Omega(v, B) = \infty$ for $v \in N$ and all $B \in \mathcal{B}$, and set $\Omega(0, \emptyset) = 0$.

Declare $(0, \emptyset)$ to be an unexplored state.

Trial State Generation

If there are no unexplored states (v, B) with $v < n$, then go to Output.

Otherwise, choose an unexplored state (v, B) , where v is as small as possible, and explore it as follows.

Use Lemma 2.2 to find the set $Q = N \setminus R$ of unscheduled jobs. Set $q = \min\{j | j \in Q\}$.

For each $k \in Q$ such that $k = q$ or $E_k(\Omega(v, B), B) - p_k < e_q - p_q$, where $E_k(\Omega(v, B), B)$ is obtained from (1), generate a trial state by appending job k . Assign the label $(v + 1, B, Q \setminus \{k\}, E_k(\Omega(v, B), B))$ to the trial state.

Trial State Extension

For each trial state (v', B, Q', E') :

If there is a job $j \in Q'$ where $e_j - p_j \leq E'$, then replace (v', B, Q', E') with the new extended trial state $(v' + 1, B, Q' \setminus \{j\}, E' + p_j)$, and set $v' = v' + 1$, $Q' = Q' \setminus \{j\}$ and $E' = E' + p_j$.

Repeat the Trial State Extension step until all trial states are nonextendable.

Trial State Elimination

For each nonextendable trial state (v', B, Q', E') :

Compute the updated buffer configuration $B' = \{j | j \in B, e_j > E'\}$ at time E' . If $E' < \Omega(v', B')$, then set $\Omega(v', B') = E'$ and declare (v', B') to be an unexplored state.

Go to Trial State Generation.

Output

Compute the optimal makespan, $\Omega(n, \emptyset)$ if $\Omega(n, \emptyset) < \infty$, and $\max\{e_1, \dots, e_n\}$ otherwise. Backtrack to find the corresponding schedule. Output the schedule and its makespan.

In accordance with Lemma 2.1, the jobs that can be chosen in the Trial State Generation step are job q and those jobs that can start processing before time $e_q - p_q$. Each such job is considered. From Lemma 2.1, any job that satisfies the condition in the Trial State Extension step can be added to the schedule without affecting its optimality. Consequently, we only consider possible states after the completion of these two steps.

Observe that during the Trial State Generation and Trial State Extension steps, we do not record changes in the contents of the buffer. While the contents actually do change when jobs are appended, only jobs j satisfying $e_j - p_j \leq E'$ are scheduled at time E' . As a result, these changes do not need to be recorded. However, it is necessary to update the buffer configuration at the Trial State Elimination step, so that the appropriate state for comparing the value of the nonextendable partial schedule can be determined.

Theorem 2.3. *Algorithm MDP generates an optimal schedule for problem $1|b_O = b|C_{\max}$ in $O(n^{b+3})$ time.*

Proof. At each stage of Algorithm MDP, the Trial State Generation step considers every possible continuation of the current partial schedule that satisfies Lemma 2.1. The Trial State Extension step implements Lemma 2.1 so that only nonextendable partial schedules are used to update $\Omega(v', B')$. Moreover, the nonextendable partial schedules that fail to update $\Omega(v', B')$ in the Trial State Elimination step satisfy the conditions of Lemma 2.3. Consequently, Algorithm MDP finds an optimal schedule.

Since the number of possible buffer configurations is $O(n^b)$, there are $O(n^{b+1})$ states (v, B) . Thus, $O(n^{b+1})$ time is required for the Initialization step. There are $O(n)$ choices for k in the Trial State Generation step. Therefore, the time requirement for this step is $O(n^{b+2})$. In the Trial State Extension step, there are $O(n^{b+2})$ trial states. Since each trial state requires $O(n)$ time to generate a nonextendable trial state, this step requires $O(n^{b+3})$ time. Each of the $O(n^{b+2})$ nonextendable trial states in the Trial State Elimination step requires only constant time. Thus, the overall time complexity of Algorithm MDP is $O(n^{b+3})$. $\square$

We now show that problem $1|b_O|C_{\max}$ is intractable. The proof of the following result appears in Appendix A.

Theorem 2.4. *The recognition version of problem $1|b_O|C_{\max}$ is unary NP-complete.*

2.2. Sum of Dispatch Times

We now examine problems where the objective is to minimize $\sum_{j=1}^n C_j$. The results in this subsection are based on reductions from a classical scheduling problem, which has

zero earliest dispatch time for all jobs and infinite buffer capacity. This problem, which is known to be unary NP-complete, is now described.

$1|r_j'| \sum C_j$ (Lenstra et al. 1977): given n' jobs, where job j has processing time p_j' and release date r_j' for $j = 1, \dots, n'$, and an integer C' , does there exist a nonpreemptive, single machine schedule such that $\sum_{j=1}^{n'} C_j \leq C'$?

Theorem 2.5. *The recognition version of problem $1|b_O = 0| \sum C_j$ is unary NP-complete.*

Proof. Since $b_O = 0$, no job j can begin processing until time $e_j - p_j$. Thus, $1|r_j'| \sum C_j$ reduces in linear time to $1|b_O = 0| \sum C_j$ by setting $n = n'$, and $p_j = p_j'$ and $e_j = r_j' + p_j'$ for $j = 1, \dots, n$. $\square$

We next consider the cases with nonzero buffer capacity. Hall et al. (1995) present a unary NP-completeness reduction from $1|r_j'| \sum C_j$ to establish the following result.

Theorem 2.6. *The recognition versions of problems $1|b_O = b, s_j = 1| \sum C_j$, $1|b_O = b| \sum C_j$, $1|b_O, s_j = 1| \sum C_j$, and $1|b_O| \sum C_j$ are unary NP-complete.*

3. NUMBER OF LATE JOBS PROBLEMS

In this section, we examine problems where the objective is to minimize the number of late jobs or the weighted number of late jobs.

For problem $1|b_O = 0, e_j = d_j| \sum w_j U_j$, the algorithm proposed by Arkin and Silverberg (1987) for scheduling with fixed start and end times provides an optimal schedule in $O(n^2)$ time. This algorithm constructs an acyclic digraph containing $2n$ nodes that represent the start time $e_j - p_j$ and completion time e_j of each job j . There is an arc directed from the start time node $e_j - p_j$ to the completion time node e_j with weight w_j . Also, there is an arc with weight zero directed from each completion time node e_j to the start time node $e_k - p_k$ for $e_j \leq e_k - p_k$. Each directed path in the network corresponds to a set of jobs that can each be scheduled on time. A longest path (with maximum total weight) in the graph defines an optimal schedule.

For problem $1|b_O = 0, e_j = d_j| \sum U_j$, the above $O(n^2)$ algorithm for the weighted problem can be applied. However, we propose a more efficient algorithm. Since $b_O = 0$, each on-time job j must complete its processing at time e_j . We propose a greedy algorithm that selects the next on-time job to be processed so that its earliest dispatch time is as small as possible. The late jobs are scheduled arbitrarily after all of the on-time jobs are processed. In the formal statement of this algorithm, i is the index of the job currently being considered for scheduling, and t is the completion time of the current partial schedule of on-time jobs.

Algorithm GREEDY-EDD

Initialization

Index the jobs such that $e_1 \leq \dots \leq e_n$.

Set $i = 1$, $t = 0$ and $\sigma = \emptyset$.

Schedule Construction

If $t \leq e_i - p_i$, then set $\sigma = (\sigma, i)$ and set $t = e_i$.

If $i < n$, then set $i = i + 1$ and repeat the Schedule Construction step.

Output

Output $(\sigma, \bar{\sigma})$, where $\bar{\sigma}$ is an arbitrary ordering of all jobs not in σ .

Theorem 3.1. *Algorithm GREEDY-EDD generates an optimal schedule for problem $1|b_O = 0, e_j = d_j|\sum U_j$ in $O(n \log n)$ time.*

Proof. Suppose that Algorithm GREEDY-EDD selects job i to be on time, and jobs $i + 1, \dots, j$ to be late, where the jobs are indexed so that $e_1 \leq \dots \leq e_n$. It follows from the Schedule Construction step that $e_k - p_k < e_i$ for $k = i + 1, \dots, j$. Therefore, to be on time, each job $i, \dots, j$ requires some processing in the interval $[e_i - 1, e_i]$. Hence, at most one of the jobs $i, \dots, j$ can be on time. Scheduling job i to be on time is preferable to selecting one of the jobs $i + 1, \dots, j$ to be on time. This is true because the machine is available to process later jobs at least as soon if job i is processed on time. For every other block that contains one job that GREEDY-EDD selects to be on time and successive jobs that are late, we apply a similar argument. This shows that, in the Output step, σ contains the maximum number of on-time jobs. $\square$

Next, some problems, which do not have the restriction that $e_j = d_j$, are shown to be intractable. These results are based on reductions from the following classical scheduling problem, which is known to be unary NP-complete.

$1|r'_j|\sum U_j$ (Lenstra et al. 1977): given n' jobs, where job j has processing time p'_j , release date r'_j , and due date d'_j for $j = 1, \dots, n'$, and an integer C' , does there exist a non-preemptive, single machine schedule such that $\sum_{j=1}^{n'} U_j \leq C'$?

Theorem 3.2. *The recognition versions of problems $1|b_O = 0|\sum U_j$ and $1|b_O = 0|\sum w_j U_j$ are unary NP-complete.*

Proof. Since the second problem is a generalization of the first, it is sufficient to show that the recognition version of $1|b_O = 0|\sum U_j$ is unary NP-complete. The reduction from problem $1|r'_j|\sum U_j$ to problem $1|b_O = 0|\sum U_j$ is similar to that in the proof of Theorem 2.5. $\square$

Problems with a nonzero constant buffer capacity are analyzed next. We propose a dynamic programming algorithm for problem $1|b_O = b, e_j = d_j|\sum w_j U_j$, which is similar to Algorithm MDP of the previous section. Each iteration of the algorithm appends one on-time job to a current partial schedule. The states of the dynamic programming algorithm are (V, B) . In the corresponding partial schedule, V is the total weight of on-time jobs, and B is the buffer configuration when the partial schedule completes processing. The algorithm recursively computes values $\Omega(V, B)$, which represent the minimum completion time of

all partial schedules that have been generated thus far and achieve the state (V, B) .

We present two preliminary results. First, we find the set of jobs which may be appended to a partial schedule corresponding to state (V, B) . The second result establishes that the elimination criterion for the dynamic programming algorithm is valid.

Lemma 3.1. *For any partial schedule of on-time jobs corresponding to state (V, B) , which completes processing at time $\Omega(V, B)$, a job can be appended without being late if and only if it is an element of $Q = \{j | j \in N, e_j - p_j \geq \Omega(V, B)\} \setminus B$.*

Proof. Each job j , where $e_j - p_j < \Omega(V, B)$, either is processed in the partial schedule or is late. Moreover, for each job j such that $e_j - p_j \geq \Omega(V, B)$, either j is not yet scheduled or $j \in B$. Therefore, the jobs that can be appended to the partial schedule without being late correspond to the set Q . $\square$

Lemma 3.2. *Suppose two partial schedules, π_1 and π_2 , reach the same state (V, B) . Further suppose that π_1 and π_2 complete processing at this state at times $M(\pi_1)$ and $M(\pi_2)$, respectively, where $M(\pi_1) \leq M(\pi_2)$. Then, in the search for an optimal schedule, π_2 can be discarded.*

Proof. From Lemma 3.1, if Q_1 and Q_2 are the sets of jobs that can be appended to π_1 and π_2 , respectively, then $M(\pi_1) \leq M(\pi_2)$ implies that $Q_2 \subseteq Q_1$. Thus, for any schedule of on-time jobs that contains π_2 as an initial partial schedule, there is a corresponding schedule of on-time jobs that contains π_1 as an initial partial schedule. It follows that the partial schedule π_1 is preferred. $\square$

We propose a dynamic programming algorithm for problem $1|b_O = b, e_j = d_j|\sum w_j U_j$. This method is similar to Algorithm MDP for problem $1|b_O = b|C_{\max}$. However, a Trial State Extension step is not required because the elimination criterion given in Lemma 3.2 is applicable to any partial schedule, not just a nonextendable one.

Algorithm Weighted Number Late Dynamic Program (WNLDP)*Initialization*

Set $W = \sum_{j=1}^n w_j$.

Set $\Omega(V, B) = \infty$ for $V = 1, \dots, W$ and all $B \in \mathcal{B}$, set $V = 0$ and $\Omega(0, \emptyset) = 0$.

Declare $(0, \emptyset)$ to be an unexplored state.

Trial State Generation

If there are no unexplored states (V, B) , then go to Output.

Otherwise, choose an unexplored state (V, B) , where V is as small as possible, and explore it as follows.

Use Lemma 3.1 to determine the set Q of candidates for the job to be appended.

For each $k \in Q$, generate a trial state by appending job k . Assign the label $(V + w_k, B, E_k(\Omega(V, B), B))$

to the trial state, where $E_k(\Omega(V, B), B)$ is obtained from (1).

Trial State Elimination

For each trial state (V', B, E') :

Compute the updated buffer configuration $B' = \{j | j \in B, e_j > E'\}$ at time E' . If $E' < \Omega(V', B')$, then set $\Omega(V', B') = E'$ and declare (V', B') to be an unexplored state.

Go to Trial State Generation.

Output

Compute $V = \max\{V' | \min_{B \in \mathcal{B}} \{\Omega(V', B)\} < \infty\}$. Back-track to find the corresponding schedule. Output the schedule and the total weight of scheduled jobs.

Observe that the Trial State Generation step adds only one job k to any partial schedule. In accordance with Lemma 3.1, this job must satisfy $k \in Q$. Each such job is considered. Then, after updating the buffer contents, the Trial State Elimination step uses Lemma 3.2 for the possible elimination of the current, or a previously generated, partial schedule.

Theorem 3.3. *Algorithm WNLDLP generates an optimal schedule for problems $1|b_O = b, e_j = d_j | \sum w_j U_j$ and $1|b_O = b, e_j = d_j, s_j = 1 | \sum w_j U_j$ in $O(Wn^{b+1})$ time.*

Proof. Lemma 3.1 shows that Algorithm WNLDLP considers all possible schedules. The partial schedules that fail to update $\Omega(V', B')$ in the Trial State Elimination step satisfy the conditions of Lemma 3.2. Consequently, Algorithm WNLDLP finds an optimal schedule.

We next consider the time requirement of WNLDLP. Since the number of possible buffer configurations is $O(n^b)$, there are $O(Wn^b)$ states (V, B) . Thus, $O(Wn^b)$ time is required for the Initialization step. In the Trial State Generation step, there are $O(n)$ choices for k . Therefore, the time requirement for the Trial State Generation step is $O(Wn^{b+1})$. There are $O(Wn^{b+1})$ trial states in the Trial State Elimination step, each of which requires only constant time. The Output step requires $O(Wn^b)$ time. Thus, the overall time complexity of Algorithm WNLDLP is $O(Wn^{b+1})$. $\square$

Corollary 3.1. *Algorithm WNLDLP generates an optimal schedule for problems $1|b_O = b, e_j = d_j | \sum U_j$ and $1|b_O = b, e_j = d_j, s_j = 1 | \sum U_j$ in $O(n^{b+2})$ time.*

Proof. Theorem 3.3 shows that Algorithm WNLDLP generates an optimal schedule for problem $1|b_O = b, e_j = d_j | \sum w_j U_j$ in $O(Wn^{b+1})$ time. For the case of unit weights, where $W = n$, Theorem 3.3 shows that the time requirement is $O(n^{b+2})$. $\square$

Theorem 3.3 shows that problem $1|b_O = b, e_j = d_j | \sum w_j U_j$ is solvable in pseudo-polynomial time. We now show that this problem is binary NP-hard.

Theorem 3.4. *The recognition versions of problems $1|b_O = b, e_j = d_j, s_j = 1 | \sum w_j U_j$ and $1|b_O = b, e_j = d_j | \sum w_j U_j$ are binary NP-complete.*

Proof. Since the second problem is a generalization of the first, it is sufficient to consider only the first problem. Consider the input buffer problem $1|b_I = b', r'_j, d'_j = \infty, s'_j = 1 | \sum w'_j U'_j$ discussed by Nawijn et al. (1994). In this problem, there are n' jobs, and b_I denotes the capacity of the input buffer, where $b_I = b'$. Also, for each job j , r'_j is the release date, p'_j is the processing time, d'_j is the due date, and w'_j is the weight. Given an instance of $1|b_I = b', r'_j, d'_j = \infty, s'_j = 1 | \sum w'_j U'_j$, we construct an instance of $1|b_O = b, e_j = d_j, s_j = 1 | \sum w_j U_j$ by setting $n = n'$, $b = b'$, and $e_j = d_j = \max_{i \in N} \{r'_i\} + \sum_{i \in N} p'_i - r'_j$, $p_j = p'_j$, $w_j = w'_j$ and $s_j = s'_j$, for $j = 1, \dots, n$.

For any schedule, let the *antithetical* schedule contain the same jobs in reverse order. We assume without loss of generality that in the output (respectively, input) buffer problem the on-time jobs are scheduled as late (resp., early) as possible, subject to feasibility. Then, job j is waiting to be processed at time $d_j - t$ in the output buffer problem if and only if it has been completed at time $r'_j + t$ in the input buffer problem, for $0 \leq t \leq d_j$. Similarly, job j is on the machine at time $d_j - t$ in the output buffer problem if and only if it is on the machine at time $r'_j + t$ in the input buffer problem, for $0 \leq t \leq d_j$. Finally, job j is in the buffer at time $d_j - t$ in the output buffer problem if and only if it is in the buffer at time $r'_j + t$ in the input buffer problem, for $0 \leq t \leq d_j$. Thus, the constructed instance of the output buffer problem is equivalent to the given instance of the input buffer problem. Since Nawijn et al. (1994) show that the recognition version of $1|b_I = b', r'_j, d'_j = \infty, s'_j = 1 | \sum w'_j U'_j$ is binary NP-complete, then so is the recognition version of $1|b_O = b, e_j = d_j, s_j = 1 | \sum w_j U_j$. $\square$

We note that the equivalence between output and input buffer problems, which we call *reversibility*, can be used to infer algorithms as well as proofs of intractability.

The corresponding problems without the assumption that $e_j = d_j$ are unary NP-hard. The proof uses a threshold cost $C = 0$ for the constructed instance of problem $1|b_O = b, s_j = 1 | \sum U_j$, which establishes Corollary 4.1 below. For details of the proof, refer to Hall et al. (1995).

Theorem 3.5. *The recognition versions of problems $1|b_O = b, s_j = 1 | \sum U_j$, $1|b_O = b, s_j = 1 | \sum w_j U_j$, $1|b_O = b | \sum U_j$ and $1|b_O = b | \sum w_j U_j$ are unary NP-complete.*

For cases with arbitrary buffer capacity, we give the following intractability result. The proof, which appears in Appendix B, uses a constructed instance of problem $1|b_O, e_j = d_j, s_j = 1 | \sum U_j$ with a threshold cost $C = 0$, which establishes Corollary 4.2 below.

Theorem 3.6. *The recognition versions of problems $1|b_O, e_j = d_j, s_j = 1 | \sum U_j$ and $1|b_O, e_j = d_j, s_j = 1 | \sum w_j U_j$, and their generalizations in which there is no assumption that $e_j = d_j$ or $s_j = 1$, are unary NP-complete.*

4. LATENESS AND TARDINESS PROBLEMS

The problems of minimizing the maximum lateness and minimizing the total tardiness are considered in this section.

4.1. Maximum Lateness

We begin by presenting an optimality condition for all maximum lateness problems with $e_j = d_j$ for $j \in N$. Recall from Section 1.3 that job j is defined to be available at time t if $t \geq e_j - p_j$ and j has not started processing at that time.

Lemma 4.1. *For any maximum lateness problem with $e_j = d_j$ for $j \in N$, if at time t the machine is free, job i is available and $i \leq e_k$ for all unscheduled jobs k , then there exists an optimal schedule in which job i starts processing at time t .*

Proof. The proof is similar to that of Lemma 2.1. $\square$

For problem 1| $b_O = 0$, $e_j = d_j$ | $L_{\max}$, no job completes before its due date. Hoogeveen (1996) proposes an $O(n^2 \log n)$ algorithm for 1| $r_j = d_j - p_j$ | $L_{\max}$, which also solves the equivalent problem 1| $b_O = 0$, $e_j = d_j$ | $L_{\max}$.

The following result shows that the corresponding problem without the restriction that $e_j = d_j$ is intractable. The proof is based on a reduction from the following classical scheduling problem, which is known to be unary NP-complete.

1| r'_j | $L_{\max}$ (Lenstra et al. 1977): given n' jobs, where job j has processing time p'_j , release date r'_j , and due date d'_j for $j = 1, \dots, n'$, and an integer C' , does there exist a non-preemptive, single machine schedule such that $\max_{j=1, \dots, n'} \{C_j - d'_j\} \leq C'$?

Theorem 4.1. *The recognition version of problem 1| $b_O = 0$ | $L_{\max}$ is unary NP-complete.*

Proof. The proof is similar to that of Theorem 2.5. $\square$

We now present a polynomial time dynamic programming algorithm for problem 1| $b_O = b$, $e_j = d_j$ | $L_{\max}$ that is similar to Algorithm MDP of Section 2. Assume that the jobs are indexed so that $e_1 \leq \dots \leq e_n$. Each iteration of this algorithm appends one job to a current partial schedule. Then, Lemma 4.1 is applied to add available jobs until no further jobs can be added. A partial schedule, which completes processing at time t , is *nonaugmentable*, if Lemma 4.1 cannot be applied either to append jobs at time t or to delay a period of idle time that completes before time t . Observe that the conditions for Lemma 4.1 are more restrictive than those for Lemma 2.1. As a result, a partial schedule may be nonaugmentable even though some jobs are available.

The states of the dynamic programming algorithm are $(v, \bar{B}, l, B)$. As in Algorithm MDP, v is the number of jobs that are processed in the corresponding partial schedule and B is the buffer configuration when the partial schedule completes processing. If the partial schedule contains a

period of machine idle time, then l is the first job that is processed after the last period of idle time and $\bar{B}$ is the buffer configuration at the end of this last idle period. If there is no machine idle time, then $l = 0$ and $\bar{B} = \emptyset$. The dynamic programming algorithm recursively computes values $\Omega(v, \bar{B}, l, B)$, which represent the maximum lateness that is minimum among partial schedules that have been generated thus far and achieve the state $(v, \bar{B}, l, B)$.

Before proceeding with a description of our algorithm, we present two preliminary results. The first of these shows that the state variables determine which jobs are processed in the corresponding partial schedule. Also, the completion time of the partial schedule is presented. The second result establishes that the elimination criterion for the dynamic programming algorithm is valid.

Lemma 4.2. *In a nonaugmentable partial schedule corresponding to state $(v, \bar{B}, l, B)$, the set of processed jobs is $R = \{1, \dots, h\} \cup B$, where $h = v - |B|$. Further, the partial schedule completes processing at time*

$$t = \begin{cases} \sum_{j \in R} p_j & \text{if } l = 0, \\ E_l(0, \bar{B}) - p_l + \sum_{j \in R \setminus \bar{R}} p_j & \text{if } l \neq 0, \end{cases}$$

where $R = \{1, \dots, v - |B|\} \cup B$, and $\bar{R} = \{i | i \in R, e_i \leq E_l(0, \bar{B}) - p_i\} \cup \bar{B}$, and $E_l(0, \bar{B})$ is obtained from (1).

Proof. The proof that $R = \{1, \dots, h\} \cup B$ is similar to that of Lemma 2.2. Thus, R is the set of jobs which are processed in the partial schedule. Further, if the schedule contains idle time, then $\bar{R}$ is the set of jobs that are processed before the last idle time period; therefore, $R \setminus \bar{R}$ is the set of jobs that are processed after the last idle time period.

Suppose that $l = 0$. Then, the partial schedule contains no idle time, and completes processing at time $\sum_{j \in R} p_j$. Alternatively, suppose that $l \neq 0$. The expression for t now follows from the observation that immediately after the last idle time period, job l starts processing at time $E_l(0, \bar{B}) - p_l$. $\square$

Lemma 4.3. *Suppose two nonaugmentable partial schedules, π_1 and π_2 , reach the same state $(v, \bar{B}, l, B)$. Further suppose that when π_1 and π_2 complete processing at this state, they have maximum lateness values $L_{\max}(\pi_1)$ and $L_{\max}(\pi_2)$, respectively, where $L_{\max}(\pi_1) \leq L_{\max}(\pi_2)$. Then, in the search for an optimal schedule, π_2 can be discarded.*

Proof. From Lemma 4.2, the sets of jobs that have been processed in the two partial schedules are identical. This lemma also shows that both partial schedules complete processing at the same time. Thus, any feasible continuation of the second partial schedule is a feasible continuation of the first, and consequently the first partial schedule is preferred. $\square$

We propose a dynamic programming algorithm for problem 1| $b_O = b$, $e_j = d_j$ | $L_{\max}$. This algorithm has a similar structure to Algorithm MDP for problem 1| $b_O = b$ | $C_{\max}$. After a job is appended to a partial schedule,

Lemma 4.1 is applied repeatedly until a nonaugmentable partial schedule is generated. Lemma 4.3 provides an elimination criterion which is applied to the resulting nonaugmentable partial schedules.

Algorithm Maximum Lateness Dynamic Program (MLDP)

Initialization

Set $\Omega(v, \bar{B}, l, B) = \infty$ for $v \in N, \bar{B} \in \mathcal{B}, l \in N \cup \{0\}$ and $B \in \mathcal{B}$, and set $\Omega(0, \emptyset, 0, \emptyset) = 0$.

Declare $(0, \emptyset, 0, \emptyset)$ to be an unexplored state.

Trial State Generation

If there are no unexplored states $(v, \bar{B}, l, B)$ with $v < n$, then go to Output.

Otherwise, choose an unexplored state $(v, \bar{B}, l, B)$, where v is as small as possible, and explore it as follows.

Use Lemma 4.2 to find the set $Q = N \setminus R$ of unscheduled jobs and to find the time t when the machine becomes available. Set $q = \min\{i | i \in Q\}$.

For each $k \in Q$, such that $k = q$ or $E_k(t, B) - p_k < e_q - p_q$, where $E_k(t, B)$ is obtained from (1), generate a trial state by appending job k . Set $L = \max\{\Omega(v, \bar{B}, l, B), E_k(t, B) - d_k\}$. Assign the label $(v + 1, \bar{B}', l', B, Q \setminus \{k\}, E_k(t, B), L)$ to the trial state, where $\bar{B}' = \bar{B}$ and $l' = l$ if $t + p_k = E_k(t, B)$, or $\bar{B}' = \{i | i \in B, e_i > E_k(t, B) - p_k\}$ and $l' = k$ if $t + p_k < E_k(t, B)$.

Trial State Augmentation

For each trial state $(v', \bar{B}', l', B, Q', E', L')$:

If $e_j - p_j \leq E'$ for $j = \min\{i | i \in Q'\}$, then replace $(v', \bar{B}', l', B, Q', E', L')$ with the new augmented trial state $(v' + 1, \bar{B}', l', B, Q' \setminus \{j\}, E' + p_j, \max\{L', E' + p_j - d_j\})$, and set $v' = v' + 1, Q' = Q' \setminus \{j\}, E' = E' + p_j$ and $L' = \max\{L', E' + p_j - d_j\}$.

Repeat the Trial State Augmentation step until all trial states are nonaugmentable.

Trial State Elimination

For each nonaugmentable trial state $(v', \bar{B}', l', B, Q', E', L')$:

Compute the updated buffer configuration $B' = \{j | j \in B, e_j > E'\}$ at time E' . If $L' < \Omega(v', \bar{B}', l', B')$, then set $\Omega(v', \bar{B}', l', B') = L'$ and declare $(v', \bar{B}', l', B')$ to be an unexplored state.

Go to Trial State Generation.

Output

Compute $\min_{\bar{B} \in \mathcal{B}, l \in N \cup \{0\}, B \in \mathcal{B}} \{\Omega(n, \bar{B}, l, B)\}$. Backtrack to find the corresponding schedule.

Output the schedule and its maximum lateness.

In accordance with Lemma 4.1, the jobs that can be chosen in the Trial State Generation step are job q and those jobs that can start processing before time $e_q - p_q$. Each such job is considered. From Lemma 4.1, any job that satisfies the condition in the Trial State Augmentation step can be added to the schedule without affecting its optimality. Note that the buffer is updated in Algorithm MLDP in a manner similar to that in Algorithm MDP.

Theorem 4.2. *Algorithm MLDP generates an optimal schedule for problems $1|b_O = b, e_j = d_j|L_{\max}$ and $1|b_O = b, e_j = d_j, s_j = 1|L_{\max}$ in $O(n^{2b+4})$ time.*

Proof. At each stage of Algorithm MLDP, the Trial State Generation step considers every possible continuation of the current partial schedule that satisfies Lemma 4.1. The Trial State Augmentation step implements Lemma 4.1 so that only nonaugmentable partial schedules can be used to update $\Omega(v', \bar{B}', l', B')$. Moreover, the nonaugmentable partial schedules that fail to update $\Omega(v', \bar{B}', l', B')$ in the Trial State Elimination step satisfy the conditions of Lemma 4.3. Consequently, Algorithm MLDP finds an optimal schedule.

Since the number of possible buffer configurations is $O(n^b)$, there are $O(n^{2b+2})$ states $(v, \bar{B}, l, B)$. Thus, $O(n^{2b+2})$ time is required for the Initialization step. Since there are $O(n)$ choices for k in the Trial State Generation step, the time requirement for this step is $O(n^{2b+3})$. There are $O(n^{2b+3})$ trial states in the Trial State Augmentation step. Since each trial state requires $O(n)$ time to generate a nonaugmentable state, this step requires $O(n^{2b+4})$ time. Each of the $O(n^{2b+3})$ nonaugmentable trial states in the Trial State Elimination step requires only constant time. Therefore, the overall time complexity of Algorithm MLDP is $O(n^{2b+4})$. $\square$

We conclude this section with several intractability results.

Corollary 4.1. *The recognition versions of problems $1|b_O = b, s_j = 1|L_{\max}$ and $1|b_O = b|L_{\max}$ are unary NP-complete.*

Proof. The results follow from the proof of Theorem 3.5 since these problems are generalizations of the recognition version of $1|b_O = b, s_j = 1|\Sigma U_j$ with $C = 0$. $\square$

Corollary 4.2. *The recognition version of problem $1|b_O, e_j = d_j, s_j = 1|L_{\max}$ and its generalizations in which there is no assumption that $e_j = d_j$ or $s_j = 1$ for all jobs j , are unary NP-complete.*

Proof. The results follow from the proof of Theorem 3.6 since these problems are generalizations of the recognition version of $1|b_O, e_j = d_j, s_j = 1|\Sigma U_j$ with $C = 0$. $\square$

4.2. Total Tardiness

We consider the minimization of total tardiness under three different assumptions about output buffer capacity.

Theorem 4.3. *The recognition versions of problems $1|b_O = 0, e_j = d_j|\Sigma T_j$, $1|b_O = b, e_j = d_j, s_j = 1|\Sigma T_j$ and $1|b_O, e_j = d_j, s_j = 1|\Sigma T_j$, and their generalizations in which there is no assumption that $e_j = d_j$ or $s_j = 1$ for all jobs j , are unary NP-complete.*

Proof. Observe that in each of these problems, since $C_j \geq e_j = d_j$, we have $T_j = \max\{C_j - d_j, 0\} = C_j - d_j$ for $j \in N$. Thus, the minimization of $\sum_{j=1}^n T_j$ is equivalent to the

minimization of $\sum_{j=1}^n C_j$, and the results follow from Theorems 2.5 and 2.6. $\square$

5. CONCLUDING REMARKS

This paper provides algorithms and complexity results for finite capacity output buffer versions of several of the best known single machine scheduling problems. It is interesting to note that making scheduling decisions for these problems is sometimes harder and sometimes easier than for their classical analogs with infinite buffer capacity. In a number of cases, we present algorithms that are essentially enumerative but run in polynomial time.

A number of other results are direct consequences of those contained here. Any problem that is a generalization of one of the unary *NP*-hard problems is automatically unary *NP*-hard. Some examples are the recognition versions of problems $1|b_O = 0|\sum w_j C_j$, $1|b_O = 0|\sum w_j T_j$, and $1|b_O = 0, r_j|\sum C_j$.

We believe that our work has practical implications for the design of manufacturing systems. That is, we have identified circumstances under which the provision of a finite capacity output buffer can simplify or complicate scheduling decisions. This may lead to recommendations for adding or removing buffer capacity, depending upon current patterns of buffer usage and also upon overall scheduling objectives.

Considerable future work is needed to improve both the order of complexity and the computational efficiency of the polynomial time, enumerative algorithms described here. In their present form, it is not clear to what extent they can provide solutions for industrial size scheduling problems. There are also a large number of finite capacity output buffer scheduling problems for which the computational complexity is unknown. These include problems that allow job preemptions. The design and analysis of heuristics for problems which are now known to be *NP*-complete is another area for further research.

There are also open questions concerning various restrictions of the problems that we consider. As an example, little is known about the complexity of problems with unit processing times or common earliest dispatch times. Also, there has been no work on output buffer problems in which dispatching a job from the buffer interrupts processing. There are also important problems related to input buffers, as discussed by Hall et al. (1997, 1998), and nothing is known about problems with both input and output buffers. We hope that our work will stimulate research in these directions.

APPENDIX A

Proof of Theorem 2.4. We use a reduction from the following problem, which is known to be unary *NP*-complete.

3-Partition (Garey and Johnson 1979): given $3m$ elements with integer sizes $a_1, \dots, a_{3m}$, where $\sum_{i=1}^{3m} a_i = my$ and $y/4 < a_i < y/2$ for $i = 1, \dots, 3m$, does there exist a

partition $S_1, \dots, S_m$ of the index set $\{1, \dots, 3m\}$ such that $|S_j| = 3$ and $\sum_{i \in S_j} a_i = y$ for $j = 1, \dots, m$? We assume without loss of generality that, if there exists a solution to 3-Partition, then the elements are numbered such that $a_{3i-2} + a_{3i-1} + a_{3i} = y$ for $i = 1, \dots, m$.

Consider the following instance of $1|b_O|C_{\max}$:

$$\begin{aligned} n &= 5m, \\ p_j &= s_j = a_j, \quad j = 1, \dots, 3m, \\ e_j &= C, \quad j = 1, \dots, 3m, \\ p_j &= (4m + 1 - j)K, \quad j = 3m + 1, \dots, 4m, \\ e_j &= (j - 3m)[K^3 + (5m + 1 - j)K/2 + y], \\ &\quad j = 3m + 1, \dots, 4m, \\ s_j &= (6m - j)y, \quad j = 3m + 1, \dots, 4m, \\ p_j &= K^3, \quad j = 4m + 1, \dots, 5m, \\ e_j &= (j - 4m)[K^3 + (6m + 1 - j)K/2 + y], \\ &\quad j = 4m + 1, \dots, 5m, \\ s_j &= 3my + 1, \quad j = 4m + 1, \dots, 5m, \\ b_O &= 3my, \end{aligned}$$

where $C = m[K^3 + (m + 1)K/2 + y]$, and $K = \sum_{j=1}^{3m} a_j + 1$.

We prove that there exists a schedule for this instance of $1|b_O|C_{\max}$ with $C_{\max} \leq C$ if and only if there exists a solution to 3-Partition. Let $J_1 = \{1, \dots, 3m\}$, $J_2 = \{3m + 1, \dots, 4m\}$, and $J_3 = \{4m + 1, \dots, 5m\}$. Note that jobs of J_1 are created from the 3-Partition data and have a common earliest dispatch time that is equal to the sum of the processing times of all n jobs, C . The last job of J_2 and of J_3 are not necessary for the proof, but are included for notational convenience.

(f) Consider the schedule

$$\begin{aligned} \sigma &= (1, 2, 3, 3m + 1, 4m + 1, \dots, \\ &\quad 3k - 2, 3k - 1, 3k, 3m + k, 4m + k, \dots, \\ &\quad 3m - 2, 3m - 1, 3m, 4m, 5m). \end{aligned}$$

The first group of five jobs of σ consists of three jobs of J_1 , one of J_2 and one of J_3 . Because a solution to 3-Partition exists, the three jobs of J_1 have a total processing time of y . The total processing time of these five jobs is $K^3 + mK + y$. After the processing of the job from J_2 , the buffer is full. However, the job of J_3 completes processing exactly at its earliest dispatch time. Then the processed jobs of J_2 and J_3 are dispatched. Each subsequent group of five jobs is processed similarly. Since this schedule is feasible and has no idle time, it is optimal. Figure 1 shows this solution where $C_{\max} = m[K^3 + (m + 1)K/2 + y]$.

(d) Note that since $C = \sum_{j=1}^{5m} p_j$, no idle time is possible in a schedule with $C_{\max} \leq C$. We show that, in such a schedule, the set of jobs processed in the interval $[(j - 3m)[K^3 + (5m + 1 - j)K/2 + y], (j - 3m + 1)[K^3 + (5m - j)K/2 + y]]$ for $j = 3m, \dots, 4m - 1$ must contain three jobs of J_1 , one job of J_2 , and one job of J_3 .

We first consider the interval $[0, K^3 + mK + y]$. Since $K = my + 1$, we have that $K^3 > \sum_{j \in J_1 \cup J_2} p_j$. Because the

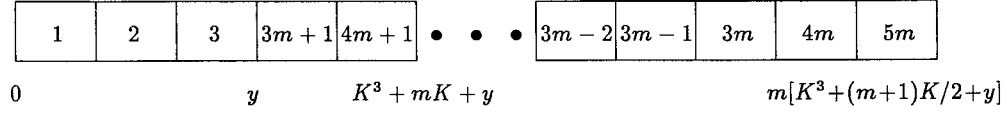


Figure 1. Optimal schedule in Theorem 2.4.

schedule has no idle time, there must be at least one element of J_3 that starts processing in $[0, K^3 + mK + y]$. Job $4m + 1$ is the only element of J_3 processed in this interval because $s_j > b_O$ for $j \in J_3$. For the same reason, this job must complete processing no earlier than time $K^3 + mK + y$.

To occupy the remaining time of at least $mK + y$ in $[0, K^3 + mK + y]$, at least one job of J_2 is needed. Since $s_i + s_j > b_O$ for $i, j \in J_2$, at most two jobs of J_2 can start processing in this interval. Suppose that two jobs of J_2 are processed in $[0, K^3 + mK + y]$. Because job $4m + 1$ completes processing no earlier than $K^3 + mK + y$, the second job from J_2 must complete processing before that time. In this case, both jobs from J_2 would be in the buffer simultaneously, a contradiction. Thus, only one job from J_2 can start processing before $K^3 + mK + y$. This job must be $3m + 1$, since no other job in J_2 has a processing time large enough to prevent idle time.

Since $p_{3m+1} = mK$, the remaining time of at least y in $[0, K^3 + mK + y]$ must be filled by jobs of J_1 . If the processing time of these jobs exceeds y , then, since $s_i = p_i$ for $i \in J_1$, these jobs and the job of J_2 cannot all fit in the buffer. If the processing time of these jobs is less than y , then there is idle time because job $4m + 1$ cannot start before time $mK + y$. Consequently, the processing time of the jobs of J_1 in the interval must equal y . Since $y/4 < p_i < y/2$ for $i \in J_1$, there are exactly three jobs of J_1 in $[0, K^3 + mK + y]$. Observe that the four jobs of J_1 and J_2 can be processed in any order, and that the job of J_3 must be processed last to complete at time $K^3 + mK + y$.

This analysis can be repeated for each subsequent interval $[(j - 3m)[K^3 + (5m + 1 - j)K/2 + y], (j - 3m + 1)[K^3 + (5m - j)K/2 + y]]$ for $j = 3m + 1, \dots, 4m - 1$. Consequently, each interval has three jobs of J_1 , and those three jobs have a total processing time of y . This implies the existence of a solution to 3-Partition.

APPENDIX B

Proof of Theorem 3.6. Since the other problems are a generalization of the first one, it is sufficient to show that the recognition version of $1|b_O, e_j = d_j, s_j = 1|\sum U_j$ is unary NP-complete. We use a reduction from 3-Partition. Consider the following instance of $1|b_O, e_j = d_j, s_j = 1|\sum U_j$:

$$\begin{aligned} n &= 3m + (m - 1) + \sum_{i=1}^{m-1} 3i, \\ p_j &= 2a_j, \quad j = 1, \dots, 3m, \\ e_j &= 2ym + (m - 1) + \sum_{i=1}^{m-1} 3i, \quad j = 1, \dots, 3m, \\ p_j &= 1, \quad j = 3m + 1, \dots, n, \end{aligned}$$

$$\begin{aligned} e_j &= 2yk + k + \sum_{i=1}^k 3(m - i), \\ j &= 3m + k, \quad k = 1, \dots, m - 1, \\ e_j &= 2yk + (k - 1) + \sum_{i=1}^k 3(m - i), \\ j &= 4m + \sum_{i=1}^{k-1} 3(m - i), \dots, 7m - 1 - 3k \\ &+ \sum_{i=1}^{k-1} 3(m - i), \quad k = 1, \dots, m - 1, \end{aligned}$$

$$b_O = 3m - 1,$$

$$C = 0.$$

Let $J_1 = \{1, \dots, 3m\}$, $J_2 = \{3m + 1, \dots, 4m - 1\}$, and $J_3 = \{4m, \dots, n\}$. The first $3m - 3$ jobs of J_3 have a due date $2y + 3m - 3$, and one job of J_2 has a due date $2y + 3m - 2$. Next, $3m - 6$ jobs of J_3 have a due date $4y + 1 + (3m - 3) + (3m - 6)$ and one job of J_2 has a due date $4y + 2 + (3m - 3) + (3m - 6)$. At the $(m - 1)$ th repetition, three jobs of J_3 have a due date $2y(m - 1) + (m - 2) + \sum_{i=1}^{m-1} 3(m - i)$ and one job of J_2 has a due date $2y(m - 1) + (m - 1) + \sum_{i=1}^{m-1} 3(m - i)$.

We prove that there exists a schedule for this instance of $1|b_O, e_j = d_j, s_j = 1|\sum U_j$ with $\sum_{j=1}^n U_j \leq C$ if and only if there exists a solution to 3-Partition.

(f) Consider the schedule

$$\begin{aligned} \sigma &= \left(4m, \dots, 4m - 1 + \sum_{i=1}^1 3(m - i), 1, 2, 3, 3m + 1, \right. \\ &\quad 4m + \sum_{i=1}^1 3(m - i), \dots, 4m - 1 \\ &\quad + \sum_{i=1}^2 3(m - i), 4, 5, 6, 3m + 2, \dots, \\ &\quad 4m + \sum_{i=1}^{k-1} 3(m - i), \dots, 4m - 1 \\ &\quad + \sum_{i=1}^k 3(m - i), 3k - 2, 3k - 1, 3k, 3m + k, \\ &\quad \dots, 4m + \sum_{i=1}^{m-2} 3(m - i), \dots, 4m - 1 \\ &\quad + \sum_{i=1}^{m-1} 3(m - i), 3m - 5, 3m - 4, \\ &\quad \left. 3m - 3, 4m - 1, 3m - 2, 3m - 1, 3m \right). \end{aligned}$$

In σ there is a repeating sequence of several jobs of J_3 , then three jobs of J_1 , then (except at the end of the schedule) one job of J_2 . For each repeating sequence, there are

two empty places left in the buffer after the jobs of J_3 are processed. Just as the third job of J_1 completes processing, all of the processed J_3 jobs are removed from the buffer and dispatched. Then the job of J_2 is processed and just meets its due date. This schedule is feasible and has $\sum_{j=1}^n U_j = 0$.

(d) Since $\sum_{j=1}^n p_j = \max_{1 \leq j \leq n} \{e_j\}$ and $e_j = d_j$ for $j = 1, \dots, n$, no idle time is possible in a schedule with $\sum_{j=1}^n U_j = 0$. Because $\sum_{j=1}^n U_j = 0$, jobs $4m, \dots, 4m - 1 + 3(m - 1)$ must complete processing by time $2y + 3m - 3$. At time $2y + 3m - 4$, the buffer must therefore contain at least $3m - 4$ of these $3m - 3$ jobs.

Therefore, the processing time of $2y$ in the interval $[0, 2y + 3m - 3]$ that is not used by jobs $4m, \dots, 4m - 1 + 3(m - 1)$ must be used to process no more than three jobs, or the buffer capacity is exceeded. Suppose at most two of these jobs (i and j) are from J_1 . Since $p_k = 2a_k < y$ for $k \in J_1$, we have that $p_i + p_j \leq 2y - 2$ for $i, j \in J_1$. Thus, idle time results. Consequently, the processing time of $2y$ is used by three jobs which must come from J_1 . If the total processing time of those three jobs is less than $2y$, then there is idle time on the machine. If the processing time exceeds $2y$, then job $3m + 1$ is late, and $\sum_{j=1}^n U_j > C$. Therefore, the total processing time of the three jobs is equal to $2y$.

A similar argument can be used at each due date for the jobs of $J_2 \cup J_3$. Thus, $\sum_{j=1}^n U_j = 0$ implies the existence of a solution to 3-Partition.

ACKNOWLEDGMENTS

This research is supported in part by the Summer Fellowship Program (Fisher College of Business, The Ohio State University). The authors are grateful to two referees for constructive suggestions which helped improve the presentation of the results.

REFERENCES

- ARKIN, E. M. AND E. B. SILVERBERG. 1987. Scheduling Jobs with Fixed Start and End Times. *Discrete Appl. Math.* **18**, 1–8.
- GAITHER, N. 1990. *Production and Operations Management*. Fourth Edition. The Dryden Press, Chicago, IL.
- GAREY, M. R. AND D. S. JOHNSON. 1979. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. Freeman, San Francisco, CA.
- GRAHAM, R. L., E. L. LAWLER, J. K. LENSTRA, AND A. H. G. RINNOOY KAN. 1979. Optimization and Approximation in Deterministic Machine Scheduling: A Survey. *Ann. Discrete Math.* **5**, 287–326.
- HALL, N. G., M. E. POSNER, AND C. N. POTTS. 1995. Scheduling with Finite Capacity Output Buffers. Working Paper, The Ohio State University.
- HALL, N. G., M. E. POSNER, AND C. N. POTTS. 1997. Preemptive Scheduling with Finite Capacity Input Buffers. *Ann. O. R.* **70**, 399–413.
- HALL, N. G., M. E. POSNER, AND C. N. POTTS. 1998. Scheduling with Finite Capacity Input Buffers. *Opns. Res.* **46**, this issue.
- HALL, N. G. AND C. SRISKANDARAJAH. 1996. A Survey of Machine Scheduling Problems with Blocking and No-wait in Process. *Opns. Res.* **44**, 510–525.
- HOOGVEEN, J. A. 1996. Minimizing Maximum Promptness and Maximum Lateness on a Single Machine. *Math. O. R.* **21**, 100–114.
- KARP, R. M. 1972. Reducibility Among Combinatorial Problems. In *Complexity of Computer Computations*. R. E. Miller and J. W. Thatcher (eds.), Plenum Press, New York.
- LAWLER, E. L. AND J. M. MOORE. 1969. A Functional Equation and Its Application to Resource Allocation and Sequencing Problems. *Mgmt. Sci.* **16**, 77–84.
- LENSTRA, J. K., A. H. G. RINNOOY KAN, AND P. BRUCKER. 1977. Complexity of Machine Scheduling Problems. *Ann. Discrete Math.* **1**, 343–362.
- MCCLAINE, J. O. AND L. J. THOMAS. 1985. *Operations Management*. Second Edition, Prentice-Hall, Englewood Cliffs, NJ.
- NAWIJN, W. M. 1992. Minimum Loss Scheduling Problems. *Euro. J. Opl. Res.* **56**, 364–369.
- NAWIJN, W. M., W. KERN, AND S. BAAS. 1994. Scheduling Jobs with Release Times on a Machine with Finite Storage. *Euro. J. Opl. Res.* **74**, 120–127.
- RAMUDHIN, A. AND H. D. RATLIFF. 1991. Scheduling in Process Industries. Working Paper, Université du Québec à Trois-Rivières, Québec, Canada.
- REKLAITIS, G. V. 1982. Review of Scheduling of Process Operations. *A.I.Ch.E. Symposium Series*, **78**, 119–133.
- WEISS, H. J. AND M. E. GERSHON. 1993. *Production and Operations Management*. Second Edition, Allyn and Bacon, Boston, MA.